

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления и мониторинга
CI/CD-пайплайнов»

Выполнил: Мижитдоржиев Гэлэг Намдакович
Группа: 241-327

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«_____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Мижитдоржиеву Гэлэгу Намдаковичу,

обучающемуся группы 241-327,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления и мониторинга CI/CD-пайплайнов»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.
2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих CI/CD-систем и инструментов автоматизации	10	22.03.2026	
Задача 1.2. Анализ программных средств разработки и обоснование технологического стека	7	26.03.2026	
Задача 1.3. Формулировка целей, задач и технических требований к веб-приложению	8	30.03.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории и вариантов использования (Use Case, User Story)	6	04.04.2026	
Задача 2.2. Проектирование модели данных (ER-диаграмма, физическая схема PostgreSQL)	7	09.04.2026	
Задача 2.3. Проектирование архитектуры безопасности (OAuth2, JWT, шифрование Fernet)	6	14.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.4. Разработка макетов пользовательского интерфейса (Wireframe)	6	18.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка асинхронного REST API на FastAPI и ORM-моделей БД	8	25.04.2026	
Задача 3.2. Реализация модуля аутентификации (JWT) и подсистемы безопасного хранения токенов провайдеров	7	01.05.2026	
Задача 3.3. Разработка сервиса интеграции с GitHub Actions API и синхронизации пайплайнов	8	07.05.2026	
Задача 3.4. Реализация диспетчеризации сборок (Re-run) и аналитического модуля расчета KPI	9	13.05.2026	
Задача 3.5. Разработка клиентского интерфейса на React (SPA) и интеграция с бэкендом	8	19.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Контейнеризация системы (Docker Compose, Nginx) и публикация репозитория	4	22.05.2026	
Задача 4.2. Развертывание сервисов на облачной платформе Render	3	24.05.2026	
Задача 4.3. Оформление расчетно-пояснительной записки	3	26.05.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«30» марта 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

«30» марта 2026 г.

Дата сдачи выполненной работы

«22» сентября 2026 г.

Задание принял к исполнению

«30» марта 2026 г.



(подпись)

Г. Н. Мижитдор-
жиев

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	8
1.1 Обзор существующих CI/CD-систем и инструментов автоматизации	8
1.2 Анализ программных средств разработки и выбор технологического стека	10
1.3 Формулировка цели, задач и требований проекта	13
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	16
2.1 Анализ целевой аудитории	16
2.2 Описание функциональности приложения	17
2.2.1 Диаграмма вариантов использования	17
2.2.2 Пользовательские истории (User Stories)	19
2.3 Проектирование модели данных	19
2.3.1 Концептуальная и логическая структура базы данных .	20
2.3.2 Физическая схема базы данных	20
2.4 Разработка макетов страниц (Wireframe)	23
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	25
3.1 Разработка базовой структуры приложения и шаблонов страниц	25
3.2 Реализация аутентификации и авторизации пользователей . .	26
3.2.1 Серверная логика безопасности	26
3.2.2 Клиентская интеграция аутентификации	27
3.3 Реализация CRUD-интерфейса для управления пайплайнами и этапами выполнения	28
3.3.1 Шифрование конфиденциальных данных	28
3.3.2 Спецификация маршрутов REST API	28
3.4 Реализация системы запуска, мониторинга и отображения статусов CI/CD-пайплайнов	29
3.4.1 Асинхронные адаптеры интеграции	29
3.4.2 Отображение состояния сборок на клиентской части .	30

3.5	Реализация системы логирования, хранения и обработки данных запусков	31
3.5.1	Сбор и нормализация журналов выполнения	31
3.5.2	Агрегация и визуализация аналитических метрик . . .	31
ЗАКЛЮЧЕНИЕ		33

ВВЕДЕНИЕ

В современных условиях разработка программного обеспечения требует высокой скорости выпуска обновлений и постоянного контроля качества. Для решения этих задач команды используют практики непрерывной интеграции и непрерывной доставки, позволяющие автоматически запускать сборку, тестирование и развертывание приложений после изменений в исходном коде.

При росте количества проектов, окружений и команд CI/CD-процессы становятся сложнее. Информация о пайплайнах может находиться в разных системах: GitHub Actions, GitLab CI/CD, Jenkins, Argo CD и других инструментах. Разработчикам и DevOps-инженерам приходится переключаться между несколькими интерфейсами, вручную отслеживать статусы запусков, искать причины ошибок в логах и контролировать готовность релизов. Это увеличивает время реакции на сбои и снижает прозрачность процесса разработки.

Актуальность темы обусловлена необходимостью создания единого веб-интерфейса, который позволит централизованно управлять CI/CD-пайплайнами и отслеживать их состояние. Такое приложение может использоваться для просмотра проектов, запусков, этапов выполнения, логов, истории статусов и результатов сборки. Наличие единой панели мониторинга упрощает контроль качества и помогает быстрее выявлять проблемные участки процесса поставки программного обеспечения.

Цель курсового проекта — разработать веб-приложение для управления и мониторинга CI/CD-пайплайнов.

Для достижения цели необходимо решить следующие задачи:

- проанализировать предметную область и существующие CI/CD-инструменты;
- выбрать программные средства разработки и технологический стек;
- сформулировать требования к веб-приложению;
- спроектировать структуру приложения и модель данных;
- реализовать основные функции управления проектами, пайплайнами, запуском и просмотром логов;
- провести проверку работоспособности основных сценариев.

Объектом исследования являются процессы автоматизации сборки, тестирования и развертывания программного обеспечения. Предметом исследования является веб-приложение, обеспечивающее управление и мониторинг CI/CD-пайплайнов.

Курсовой проект состоит из введения, аналитического раздела, разделов проектирования и реализации, заключения и списка использованных источников. В первом разделе рассматриваются существующие CI/CD-системы, проводится выбор технологического стека и формулируются требования к разрабатываемому приложению.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих CI/CD-систем и инструментов автоматизации

Современная разработка программного обеспечения все чаще строится вокруг практик непрерывной интеграции и непрерывной доставки (CI/CD). CI/CD-пайплайн представляет собой формализованную последовательность этапов, в которой исходный код автоматически проверяется, собирается, тестируется и подготавливается к развертыванию. Такой подход снижает количество ручных операций, ускоряет выпуск новых версий и делает процесс поставки программного продукта более прозрачным и контролируемым.

Для инженерных команд критически важны не только внутренние механизмы исполнения сборок, но и удобные средства мониторинга и оперативного управления: централизованное отображение текущих запусков, контроль сбоев, анализ логов, возможность ручного перезапуска упавших задач и оценка стабильности процессов. При использовании микросервисной архитектуры и распределении сервисов по десяткам репозиторий стандартные интерфейсы отдельных CI/CD-систем приводят к фрагментации контроля. Возникает потребность в специализированном веб-приложении, агрегирующем данные из внешних сервисов автоматизации в концепции единого окна («Single Pane of Glass»).

В рамках анализа предметной области рассмотрены наиболее распространенные решения: GitHub Actions, GitLab CI/CD, Jenkins и Argo CD.

GitHub Actions — встроенная в платформу GitHub система автоматизации, в которой сценарии описываются в виде workflow-файлов формата YAML. Система глубоко интегрирована с событиями репозитория: pull requests, коммитами, релизами и внутренним хранилищем секретов. Данное решение оптимально для проектов, размещенных на GitHub [1]. Однако его ключевым ограничением является жесткая привязка к экосистеме GitHub, что затрудняет сквозной мониторинг гетерогенной инфраструктуры.

GitLab CI/CD — компонент комплексной DevOps-платформы GitLab, конфигурируемый через файл `.gitlab-ci.yml`. Решение предоставляет развитые возможности по управлению стадиями, зависимостями задач, собственными runner-агентами, переменными окружения и артефактами [2]. Главным достоинством является полнота экосистемы, а ограничением — ориента-

ция на закрытый контур GitLab, требующая отдельной настройки внешних интеграций через REST API и webhooks.

Jenkins — классическая расширяемая система автоматизации с открытым исходным кодом. Механизм Jenkins Pipeline позволяет описывать сложные сценарии сборки в файлах Jenkinsfile [3]. Система независима от Git-хостинга и обладает обширной экосистемой плагинов. К ее недостаткам относятся высокая ресурсоемкость, сложность администрирования и визуальная неоднородность интерфейса.

Argo CD — декларативный инструмент GitOps-доставки приложений в среду Kubernetes. Argo CD отслеживает состояние манифестов в Git-репозитории и синхронизирует их с фактическим состоянием кластера [4]. Данный инструмент не предназначен для выполнения этапов компиляции и тестирования, а используется на финальной стадии доставки совместно с традиционными CI-системами.

Сравнительная характеристика рассмотренных инструментов автоматизации приведена в таблице 1.1.

Таблица 1.1 – Сравнительный анализ CI/CD-систем

Система	Основное назначение	Сильные стороны	Ограничения	Значимость для проекта
GitHub Actions	Автоматизация workflow внутри GitHub	Простая настройка, развитая экосистема actions, тесная связь с PR	Привязка к GitHub, лимиты бесплатного времени runner	Источник данных для интеграции через GitHub REST API
GitLab CI/CD	Полный цикл CI/CD в среде GitLab	Единая платформа, runner-агенты, артефакты, environments	Зависимость от инфраструктуры GitLab	Источник данных для интеграции через GitLab API
Jenkins	Универсальная система автоматизации	Предельная гибкость, независимость от хостинга кода	Сложность поддержки, устаревший базовый интерфейс	Потенциальный источник интеграции внешних jobs
Argo CD	GitOps-доставка в Kubernetes	Синхронизация состояний, надежный деплой в кластер	Не выполняет этапы сборки и модульного тестирования	Пример специализированного инструмента доставки

Проведенный анализ показывает, что инструменты CI/CD эффективно функционируют внутри своих экосистем, однако в командах с гетерогенной инфраструктурой инженерам приходится постоянно переключаться между различными панелями управления.

Разрабатываемое веб-приложение призвано решить эту проблему в роли централизованного агрегатора и пульта управления: оно должно собирать актуальные статусы сборок через API внешних систем, хранить историю запусков, предоставлять быстрый доступ к логам и поддерживать диспетчеризацию (повторный запуск упавших пайплайнов) из единого интерфейса.

1.2 Анализ программных средств разработки и выбор технологического стека

Для реализации веб-приложения мониторинга и диспетчеризации пайплайнов необходим технологический стек, обеспечивающий высокую отзывчивость интерфейса, эффективную асинхронную обработку внешних сетевых запросов и безопасное хранение учетных записей и токенов интеграций.

Архитектура системы строится на разделении клиентской части (Single Page Application, SPA) и серверной части (REST API), взаимодействующих по протоколам HTTP/JSON и WebSocket.

Клиентская часть. Интерфейс системы мониторинга должен в реальном времени отображать статусы большого количества параллельных процессов, предоставлять средства динамической фильтрации, поиска по веткам и коммитам, а также отрисовывать графики стабильности сборок. Для разработки клиентского приложения рассмотрены технологии React, Vue.js и серверная генерация HTML (Jinja/Blade).

Таблица 1.2 – Сравнение фронтенд-технологий

Критерий	React (выбран)	Vue.js	Jinja (серверный рендеринг)
Архитектурный подход	SPA на базе компонентов и Virtual DOM [5]	SPA на базе реактивных компонентов	Серверная генерация статических страниц
Динамичность интерфейса	Мгновенное обновление карточек статусов без перезагрузки	Быстрое реактивное обновление DOM-дерева	Требует полной перезагрузки страницы или сложных скриптов
Экосистема для дашбордов	Огромный выбор библиотек (Lucide, Recharts, Tailwind UI)	Достаточный выбор библиотек компонентов	Ограничена базовыми CSS-фреймворками
Разделение ответственности	Полная изоляция логики представления от бэкенда	Полная изоляция клиентской части	Жесткая связность с кодом серверного фреймворка

В качестве основы фронтенда выбрана библиотека **React**. Компонентный подход позволяет изолировать логику отображения отдельных карточек пайплайнов, индикаторов статусов и модальных окон управления. Использование Virtual DOM и хуков состояния обеспечивает высокую производительность при частых обновлениях данных через WebSockets без необходимости перерисовки всего экрана. Сборка проекта осуществляется с использованием современного инструмента Vite.

Серверная часть. Сервер приложения выполняет функции шлюза и агрегатора: принимает HTTP-запросы от клиентской части, осуществляет фоновый опрос и прием webhooks от внешних платформ (GitHub Actions, GitLab CI/CD), сохраняет промежуточные снапшоты в БД и транслирует команды повторного запуска (re-run). Для серверной реализации рассмотрены фреймворки на языке Python: FastAPI, Flask и Django.

- **FastAPI (выбран):** современный высокопроизводительный асинхронный фреймворк, построенный на базе ASGI-сервера Starlette и библиотеки валидации данных Pydantic [6]. Ключевым преимуществом FastAPI является нативная поддержка корутин (async/await), что критически важно для приложения-агрегатора, совершающего множество параллельных

сетевых I/O-запросов к внешним API без блокировки основного потока. Кроме того, фреймворк автоматически генерирует интерактивную OpenAPI-документацию (Swagger UI).

- **Flask:** классический микрофреймворк с синхронной архитектурой (WSGI). Реализация высоконагруженного асинхронного взаимодействия с внешними API и работа с протоколом WebSocket во Flask требуют подключения дополнительных надстроек, что усложняет архитектуру.
- **Django:** полнофункциональный монолитный фреймворк. Включает избыточный для компактного API-сервиса функционал (административную панель, встроенный ORM с привязкой к синхронным вызовам), что создает лишние накладные расходы при разработке легковесного шлюза.

С учетом необходимости асинхронного ввода-вывода и строгой типизации контрактов API в качестве серверного фреймворка выбран **FastAPI**.

Хранение данных. Для долговременного хранения сведений о зарегистрированных пользователях, проектах, конфигурациях токенов доступа и исторических метрик сборок выбрана реляционная СУБД **PostgreSQL** [7]. Наличие поддержки типа данных JSONB позволяет сохранять неструктурированные метаданные внешних webhooks без усложнения реляционной схемы. Для взаимодействия с базой данных используется асинхронная ORM **SQLAlchemy 2.0** в связке с драйвером **asynсpg**, а миграции схемы выполняются с помощью инструмента **Alembic**.

Взаимодействие компонентов и внешняя интеграция. Связь между SPA-клиентом и бэкендом организуется через REST API в формате JSON, а для моментальной доставки сведений об изменении статуса пайплайна используются протокол WebSocket и механизм Webhooks. Сетевые запросы к API внешних провайдеров (GitHub REST API v3, GitLab REST API v4) осуществляются с использованием асинхронного HTTP-клиента **httpx**.

Сводная информация о выбранном стеке приведена в таблице 1.3.

Таблица 1.3 – Выбранный технологический стек

Компонент	Выбранная технология	Назначение
Клиентская часть	React, TypeScript, Vite	Интерактивный SPA-интерфейс дашборда, карточки пайплайнов, графики
Серверная часть	Python, FastAPI	Асинхронный REST API, диспетчеризация задач, валидация данных через Pydantic
База данных	PostgreSQL	Надежное хранение проектов, пользователей, токенов и истории запусков
ORM и миграции	SQLAlchemy 2.0 (async), Alembic	Управление реляционной схемой и асинхронный доступ к БД
Обмен данными	REST API, Webhooks, WebSocket	Взаимодействие с клиентом и интеграция с внешними CI/CD-платформами
Развертывание	Docker, Docker Compose	Контейнеризация сервисов и изоляция тестового окружения

Выбранная архитектура на базе FastAPI и React разделяет ответственность между представлением и бизнес-логикой, гарантирует высокую скорость отклика и позволяет гибко подключать новых CI/CD-провайдеров.

1.3 Формулировка цели, задач и требований проекта

Основная инженерная проблема заключается в фрагментации данных о сборках при одновременной разработке множества микросервисов в разных системах контроля версий. Разрабатываемое веб-приложение призвано стать единой точкой наблюдения и диспетчеризации.

Цель работы — разработать веб-приложение для централизованного управления и мониторинга CI/CD-пайплайнов, агрегирующее статусы сборок из внешних систем автоматизации и предоставляющее пользователю интерфейс оперативного реагирования и аналитики.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ существующих платформ CI/CD и выявить ключевые функциональные требования к централизованной панели мониторинга.
2. Обосновать выбор технологического стека для реализации SPA-клиента, асинхронного серверного API и хранилища данных.
3. Спроектировать архитектуру веб-приложения и реляционную модель базы данных с учетом поддержки гетерогенных внешних источников.
4. Реализовать модуль аутентификации пользователей и безопасного хранения токенов доступа к внешним сервисам.
5. Разработать программные адаптеры для интеграции с внешними API (GitHub Actions REST API, GitLab API).
6. Реализовать пользовательский интерфейс дашборда на React, обеспечивающий фильтрацию сборок, отображение карточек состояния и запуск команд повторного прогона (re-run).
7. Реализовать механизмы оперативного обновления информации (Webhooks и WebSockets).
8. Выполнить функциональное тестирование системы и оценить корректность обработки сбоев внешних сервисов.

К разрабатываемому приложению предъявляются следующие **функциональные требования**:

- аутентификация и авторизация пользователей в системе;
- управление подключениями к внешним провайдерам (ввод и безопасное хранение персональных токенов доступа);
- добавление и группировка отслеживаемых репозиторий и конвейеров сборки;
- сбор и синхронизация статусов пайплайнов (Success, Failed, Running, Queued, Canceled);
- отображение детальной информации о прогоне (автор коммита, ветка, хеш коммита, время выполнения и шаги пайплайна);

- возможность инициирования повторного запуска пайплайна (Re-run / Retry) через внешний API непосредственно из веб-интерфейса;
- фильтрация и поиск пайплайнов по статусу, проекту, автору и ветке;
- расчет базовых показателей надежности процессов (процент успешных прогонов и динамика длительности сборки).

Также сформированы **нефункциональные требования**:

- время отдачи кэшированных агрегированных данных клиентскому приложению — не более 300 мс;
- асинхронная неблокирующая архитектура взаимодействия с внешними сервисами;
- шифрование конфиденциальных данных (токенов доступа) при сохранении в базе данных;
- документирование REST API посредством автоматически генерируемой спецификации OpenAPI;
- масштабируемость архитектуры: возможность добавления новых CI/CD-провайдеров путем реализации единого программного интерфейса адаптера;
- адаптивный дизайн клиентской части для комфортной работы на экранах с различным разрешением;
- поддержка контейнеризации всех сервисов с помощью Docker Compose для быстрого развертывания.

Сформулированные требования определяют вектор дальнейшего проектирования системы: разработку диаграмм вариантов использования, проектирование базы данных и разработку макетов интерфейса в следующих разделах работы.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Эффективность проектирования веб-приложения для управления и мониторинга CI/CD-пайплайнов напрямую зависит от детального понимания потребностей будущих пользователей. В условиях командной разработки программного обеспечения с конвейерами непрерывной интеграции взаимодействуют специалисты с различными зонами ответственности.

На основе специфики предметной области выделены три ключевые группы пользователей:

1. DevOps-инженеры и системные администраторы. Специалисты отвечают за стабильность сборочной инфраструктуры, конфигурацию раннеров и доступность внешних платформ. Их ключевой интерес заключается в сквозном контроле работоспособности конвейеров, обнаружении зависших задач, анализе времени прохождения стадий и устранении инфраструктурных сбоев.
2. Разработчики программного обеспечения (Frontend, Backend, Fullstack). Основные потребители сервиса. Для них критически важно быстро получать обратную связь о статусе прохождения юнит-тестов, линтеров и сборок после отправки изменений в Git. В случае падения сборки разработчику необходим моментальный доступ к логам ошибки и возможность повторного перезапуска задачи в один клик.
3. Технические руководители и тимлиды (Team Lead, Tech Lead). Пользователи заинтересованы в высокоуровневой аналитике: общем проценте успешных сборок (Success Rate), частоте релизов, выявлении нестабильных тестов и контроле готовности релизных веток.

Для формализации требований сформированы репрезентативные профили целевых пользователей (User Personas), представленные в таблице 2.1.

Таблица 2.1 – Профили целевых пользователей (User Personas)

Персона	Контекст деятельности	Боли и проблемы	Ожидания от системы
Алексей, 28 лет, DevOps-инженер	Сопровождает 15 микросервисов, распределенных между GitHub и GitLab	Вынужден переключаться между десятком вкладок; сложно отслеживать зависшие сборки	Единая панель статусов; фильтрация по статусам; оперативные уведомления
Илья, 23 года, Backend-разработчик	Пишет код, часто делает push в feature-ветки, ждет результатов тестов	Долго ищет упавший шаг в громоздких интерфейсах; сложный путь до кнопки «Retry»	Моментальное отображение статуса коммита; лог ошибки на одном экране; быстрый перезапуск
Марина, 34 года, Team Lead	Контролирует поставку релизов и метрики качества в спринтах	Отсутствие наглядной сводки по стабильности репозитория; разрозненные отчеты	Обзорный дашборд; графики успешности сборок; аналитика среднего времени прогона

Проведенный анализ целевой аудитории показывает необходимость реализации гибкого интерфейса, предоставляющего как агрегированные сводные метрики для руководителей, так и детальные журналы сборок с кнопками управления для инженеров.

2.2 Описание функциональности приложения

Функциональная архитектура разрабатываемого приложения разделяется на модули управления учетными записями, конфигурации внешних интеграций, фонового сбора метрик и интерактивного взаимодействия с конвейерами.

2.2.1 Диаграмма вариантов использования

Для визуализации сценариев взаимодействия пользователей с системой построена диаграмма вариантов использования (Use Case diagram), представленная на рисунке 2.1.

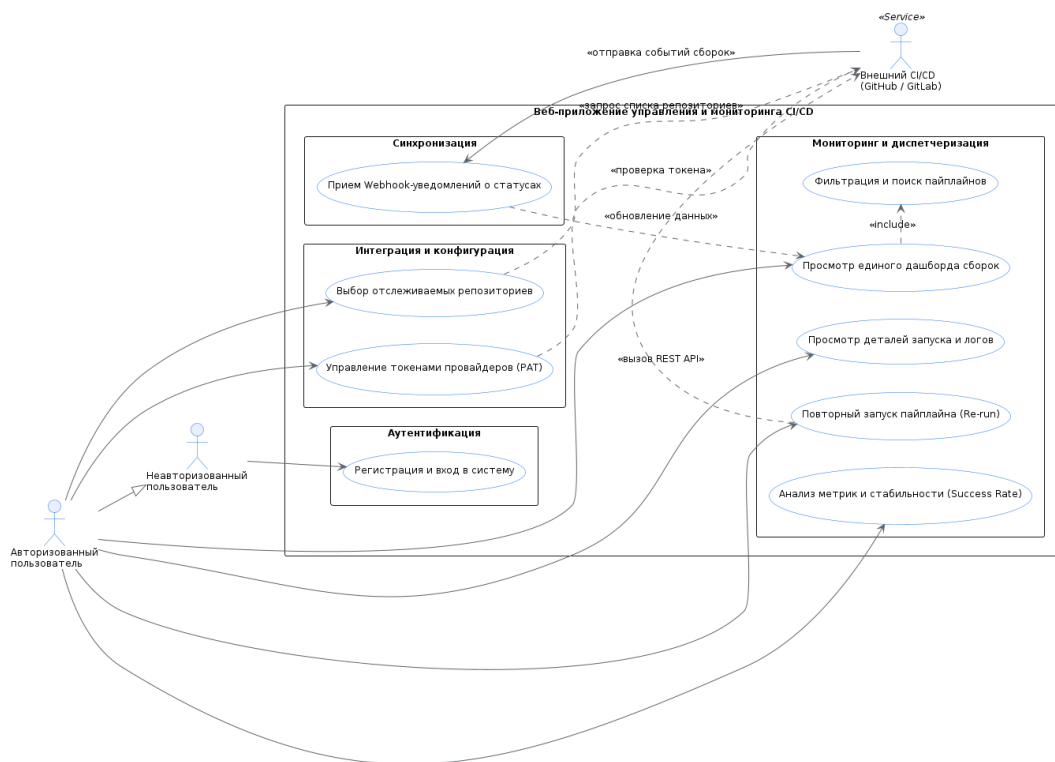


Рисунок 2.1 – Диаграмма вариантов использования веб-приложения

В системе выделены два действующих лица (актера):

- неавторизованный пользователь (гость) — имеет доступ только к формам входа и регистрации;
- авторизованный пользователь (инженер) — имеет доступ к панели мониторинга, настройкам интеграций, управлению пайплайнами и аналитическим метрикам.

Ключевыми прецедентами являются:

- управление токенами провайдеров CI/CD (GitHub PAT, GitLab Token);
- выбор и синхронизация отслеживаемых репозиторий;
- просмотр единого дашборда сборок;
- фильтрация и поиск пайплайнов по проектам, веткам и статусам;
- просмотр деталей запуска и логов выполнения сборок;
- диспетчеризация конвейера (повторный запуск упавших задач);
- анализ метрик надежности и стабильности сборок.

2.2.2 Пользовательские истории (User Stories)

Для детальной спецификации поведения системы разработаны пользовательские истории с критериями приемки (Acceptance Criteria), сведенные в таблицу 2.2.

Таблица 2.2 – Пользовательские истории веб-приложения

ID	Пользовательская история	Критерии приемки (Given/When/Then)	Ценность для пользователя
US-01	Как разработчик, я хочу подключить токен GitHub, чтобы отслеживать репозитории	Given: пользователь авторизован; When: введен корректный PAT; Then: токен зашифрован и сохранен	Безопасная интеграция без передачи основного пароля
US-02	Как инженер, я хочу видеть статусы сборок на одном экране, чтобы быстро выявлять сбои	Given: подключены провайдеры; When: открыт дашборд; Then: отображается таблица статусов	Экономия времени на переключение между системами
US-03	Как разработчик, я хочу перезапустить упавшую сборку в один клик, чтобы не заходить в GitHub	Given: сборка имеет статус Failed; When: нажата кнопка «Retry»; Then: статус сменился на Queued	Сокращение цикла исправления ошибок
US-04	Как тимлид, я хочу видеть процент успешности за неделю, чтобы оценить стабильность	Given: накоплена история запусков; When: открыт раздел метрик; Then: отображается Success Rate	Объективная оценка стабильности сборочных процессов

2.3 Проектирование модели данных

Для обеспечения надежного хранения информации, транзакционной целостности и быстрого поиска по временным меткам выбрана реляционная модель данных, реализуемая в СУБД PostgreSQL.

2.3.1 Концептуальная и логическая структура базы данных

Модель данных спроектирована с учетом разделения сущностей на конфигурационные (пользователи, провайдеры, репозитории) и операционные (пайплайны, шаги выполнения, логи).

Связи между сущностями организованы следующим образом:

- один пользователь (`users`) может сконфигурировать несколько подключений к внешним платформам (`ci_providers`) — связь «один-ко-многим» (1:N);
- каждый провайдер предоставляет доступ к множеству отслеживаемых репозиториях (`repositories`) — связь «один-ко-многим» (1:N);
- репозиторий содержит историю запусков пайплайнов (`pipelines`) — связь «один-ко-многим» (1:N);
- каждый запуск пайплайна детализируется последовательностью этапов выполнения (`pipeline_steps`) — связь «один-ко-многим» (1:N).

2.3.2 Физическая схема базы данных

Физическая схема разработана с соблюдением требований третьей нормальной формы (3NF). Для обеспечения информационной безопасности секретные ключи доступа (`encrypted_token`) хранятся в зашифрованном виде с применением симметричного алгоритма Fernet.

Спецификация структуры таблиц приведена в таблицах 2.3–2.7.

Таблица 2.3 – Спецификация таблицы пользователей (users)

Имя поля	Тип данных	Ключ	Описание и ограничения
id	INTEGER / SERIAL	PK	Уникальный идентификатор пользователя
email	VARCHAR(255)	UNIQUE	Адрес электронной почты (логин), NOT NULL
hashed_password	VARCHAR(255)	—	Хеш пароля (алгоритм bcrypt), NOT NULL
full_name	VARCHAR(150)	—	Имя и фамилия пользователя
is_active	BOOLEAN	—	Флаг активности учетной записи (default: true)
created_at	TIMESTAMPTZ	—	Дата и время регистрации (default: NOW())

Таблица 2.4 – Спецификация таблицы провайдеров CI/CD (ci_providers)

Имя поля	Тип данных	Ключ	Описание и ограничения
id	INTEGER / SERIAL	PK	Идентификатор записи интеграции
user_id	INTEGER	FK	Внешний ключ на таблицу users, ON DELETE CASCADE
provider_type	VARCHAR(32)	—	Тип сервиса: GITHUB, GITLAB
name	VARCHAR(100)	—	Пользовательское наименование подключения
api_url	VARCHAR(255)	—	Базовый URL API внешней платформы
encrypted_token	TEXT	—	Зашифрованный токен доступа (PAT), NOT NULL
is_valid	BOOLEAN	—	Статус валидности токена (default: true)

Таблица 2.5 – Спецификация таблицы репозитория (repositories)

Имя поля	Тип данных	Ключ	Описание и ограничения
id	INTEGER / SERIAL	PK	Внутренний идентификатор репозитория
provider_id	INTEGER	FK	Внешний ключ на ci_providers, ON DELETE CASCADE
external_id	VARCHAR(100)	—	Идентификатор проекта во внешней системе
full_name	VARCHAR(255)	—	Полное имя репозитория (owner/repo)
default_branch	VARCHAR(100)	—	Имя основной ветки (default: main)
web_url	VARCHAR(512)	—	Прямая веб-ссылка на репозиторий
is_monitored	BOOLEAN	—	Флаг активности мониторинга (default: true)

Таблица 2.6 – Спецификация таблицы запусков пайплайнов (pipelines)

Имя поля	Тип данных	Ключ	Описание и ограничения
id	INTEGER / SERIAL	PK	Внутренний идентификатор запуска
repository_id	INTEGER	FK	Внешний ключ на repositories, ON DELETE CASCADE
external_run_id	VARCHAR(100)	—	Идентификатор прогона во внешнем API
branch	VARCHAR(128)	—	Название целевой ветки коммита
commit_sha	VARCHAR(64)	—	Хеш-идентификатор коммита Git
commit_message	TEXT	—	Сообщение коммита
author_name	VARCHAR(150)	—	Автор изменений
status	VARCHAR(32)	—	Статус: QUEUED, RUNNING, SUCCESS, FAILED
duration_sec	INTEGER	—	Длительность выполнения в секундах
started_at	TIMESTAMPTZ	—	Время запуска сборки
finished_at	TIMESTAMPTZ	—	Время завершения выполнения

Таблица 2.7 – Спецификация таблицы стадий выполнения (pipeline_steps)

Имя поля	Тип данных	Ключ	Описание и ограничения
id	INTEGER / SERIAL	PK	Внутренний идентификатор шага
pipeline_id	INTEGER	FK	Внешний ключ на pipelines, ON DELETE CASCADE
name	VARCHAR(150)	—	Название этапа (например, Build, Test)
stage	VARCHAR(64)	—	Группа этапа в конвейере
status	VARCHAR(32)	—	Статус выполнения этапа
log_excerpt	TEXT	—	Фрагмент вывода логов при ошибке
duration_sec	INTEGER	—	Длительность выполнения этапа

Для оптимизации выполнения выборки создаются следующие индексы:

- составной B-Tree индекс idx_pipelines_repo_date по полям (repository_id, started_at DESC);
- индекс idx_pipelines_status по полю status;
- уникальный составной индекс uq_repo_external_id на поля (provider_id, external_id).

2.4 Разработка макетов страниц (Wireframe)

Пользовательский интерфейс проектируется в соответствии с концепцией Single Page Application (SPA). Главной задачей интерфейса является обеспечение высокой информативности без визуальной перегруженности, позволяя оператору за минимальное время оценить статус инфраструктуры.

В системе выделены 4 ключевых экрана:

1. Экран аутентификации (/login). Компактная форма по центру экрана, содержащая поля ввода Email и пароля, валидацию ошибок ввода и кнопку входа.
2. Главная панель мониторинга (/dashboard). Основное рабочее пространство. В верхней части расположены карточки сводных показателей, под ними находится панель фильтрации, а центральную часть занимает таблица сборов с кнопками ручного перезапуска.

3. Экран деталей запуска и логов (/pipelines/:id). Отображает последовательность стадий конвейера и окно терминала с моноширинным шрифтом для вывода логов ошибок.
4. Экран настроек интеграций (/settings/integrations). Форма для добавления новых подключений к Git-платформам и переключатели отслеживаемых репозиториев.

Схематический макет ключевого экрана мониторинга сборок (Wireframe) приведен на рисунке 2.2.

+-----+					
CI/CD Monitor & Dispatcher		[Dashboard]	[Settings]	(User)	
+-----+					
[Total: 1420]		[Success: 94%]	[Running: 3]	[Failed: 12]	
+-----+					
Filters: [All Repos v]		[Branch: main v]	[Status: All v]		
+-----+					
Repository	Branch	Commit / Author		Duration	Status
+-----+					
auth-service	main	f8a12c Fix JWT		1m 45s	[SUCCESS]
billing-api	develop	90c21b Add Stripe		3m 12s	[FAILED]
frontend-web	feature-x	e711a0 UI Update		2m 04s	[RUNNING]
+-----+					
Showing 1-3 of 84 runs				< Prev [1] 2 3 Next >	
+-----+					

Рисунок 2.2 – Схематический макет главного экрана мониторинга сборок (Wireframe)

Спроектированная архитектура интерфейса и структуры данных обеспечивает выполнение всех функциональных требований, определенных в аналитическом разделе, и формирует полную спецификацию для этапа программной реализации.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка базовой структуры приложения и шаблонов страниц

В соответствии с выбранным стеком архитектура разрабатываемого программного комплекса построена по принципу разделения клиентской части (SPA на React) и серверного интерфейса (REST API на FastAPI).

Исходный код системы организован в рамках единого репозитория с четким разграничением уровней ответственности:

- backend/ — серверная часть на языке Python:
 - app/core/ — глобальные параметры конфигурации, настройки безопасности и подключение к СУБД;
 - app/models/ — декларативные ORM-модели базы данных (SQLAlchemy 2.0);
 - app/schemas/ — схемы валидации и сериализации входящих и исходящих данных (Pydantic v2);
 - app/services/ — слой бизнес-логики: адаптеры внешних CI/CD API (GitHub, GitLab), сервис шифрования токенов и модуль диспетчеризации;
 - app/api/v1/ — модули маршрутизации HTTP-запросов (контроллеры эндпоинтов).
- frontend/ — клиентское приложение на React:
 - src/components/ — атомарные компоненты интерфейса (кнопки, индикаторы статусов, модальные окна, терминал логов);
 - src/pages/ — экранные формы приложения (страницы авторизации, главного дашборда, настроек интеграций, детальной аналитики сборок);
 - src/services/ — клиенты взаимодействия с API на базе Axios с перехватчиками (interceptors) сетевых ответов;
 - src/context/ — состояние приложения (глобальный контекст авторизации и профиля пользователя).

Маршрутизация на стороне клиента реализована с использованием библиотеки `react-router-dom`. Все защищенные экраны обернуты в компонент `ProtectedRoute`, который проверяет валидность токена сессии и выполняет перенаправление на страницу входа при его отсутствии.

Базовый каркас интерфейса (`MainLayout`) включает в себя адаптивную боковую панель навигации (`Sidebar`), верхнюю строку состояния с информацией о пользователе и центральную контентную область. Стилизация выполнена с использованием утилитарного CSS-фреймворка `Tailwind CSS`, что обеспечивает консистентный дизайн и адаптивность под различные разрешения экрана.

3.2 Реализация аутентификации и авторизации пользователей

Безопасность системы обеспечивается механизмом аутентификации на основе токенов стандарта JWT (JSON Web Token, RFC 7519) в сочетании с алгоритмом хеширования паролей `bcrypt`.

3.2.1 Серверная логика безопасности

При регистрации пользователя введенный пароль не сохраняется в открытом виде, а обрабатывается криптографической функцией хеширования с добавлением соли через библиотеку `passlib`. Для генерации цифровой подписи токенов применяется алгоритм `HMAC-SHA256` с использованием закрытого серверного ключа.

Схема реализации внедрения зависимостей (`Dependency Injection`) для защиты закрытых эндпоинтов в `FastAPI` представлена в листинге 3.1.

```

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.ext.asyncio import AsyncSession

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="/api/v1/auth/login"
)

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: AsyncSession = Depends(get_db)
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Не удалось подтвердить учетные данные",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(
            token, SECRET_KEY, algorithms=[ALGORITHM]
        )
        user_id: int = payload.get("sub")
        if user_id is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = await db.get(User, int(user_id))
    if user is None or not user.is_active:
        raise credentials_exception
    return user

```

Рисунок 3.1 – Реализация зависимости проверки JWT-токена в FastAPI

3.2.2 Клиентская интеграция аутентификации

На стороне SPA-клиента полученный JWT-токен сохраняется в защищенном хранилище браузера (localStorage). Для взаимодействия с сервером настроен экземпляр Axios, который автоматически подставляет заголовок `Authorization: Bearer <token>` во все исходящие запросы. При получении HTTP-ответа со статусом 401 (Unauthorized) перехватчик сбрасывает сессию и перенаправляет пользователя на форму входа.

3.3 Реализация CRUD-интерфейса для управления пайплайнами и этапами выполнения

Для организации мониторинга пользователь настраивает подключение к целевым Git-платформам и выбирает репозитории для отслеживания.

3.3.1 Шифрование конфиденциальных данных

Персональные токены доступа (GitHub PAT, GitLab Token) представляют собой критически важные учетные данные. Сохранение их в открытом виде недопустимо. В модуле `security.py` реализован алгоритм симметричного шифрования Fernet (на базе AES-128 в режиме CBC с аутентификацией HMAC), представленный в листинге 3.2.

```
from cryptography.fernet import Fernet
from app.core.config import settings

cipher = Fernet(settings.ENCRIPTION_KEY.encode())

def encrypt_token(plain_token: str) -> str:
    return cipher.encrypt(plain_token.encode()).decode()

def decrypt_token(encrypted_token: str) -> str:
    return cipher.decrypt(encrypted_token.encode()).decode()
```

Рисунок 3.2 – Функции шифрования и дешифрования токенов внешних провайдеров

3.3.2 Спецификация маршрутов REST API

Для управления сущностями разработаны программные эндпоинты, спецификация которых приведена в таблице 3.1.

Таблица 3.1 – Спецификация эндпоинтов управления (префикс /api/v1)

Метод	Маршрут	Назначение операции
POST	/auth/login	Аутентификация пользователя и выпуск JWT
GET	/providers	Получение списка подключенных провайдеров
POST	/providers	Подключение провайдера (с шифрованием токена)
DELETE	/providers/{id}	Удаление интеграции и связанных репозиториев
GET	/repositories	Список отслеживаемых репозиториев с фильтрацией
POST	/repositories/sync	Синхронизация списка репозиториев из внешнего API
PATCH	/repositories/{id}	Переключение флага мониторинга (is_monitored)

На клиентской стороне разработан интерфейс настроек (SettingsPage), позволяющий в интерактивном режиме вводить токены провайдеров, проверять соединение с внешним API и включать отслеживание необходимых проектов с помощью переключателей (Toggle Switch).

3.4 Реализация системы запуска, мониторинга и отображения статусов CI/CD-пайплайнов

3.4.1 Асинхронные адаптеры интеграции

Для взаимодействия с внешними системами автоматизации разработан сервисный уровень с базовым интерфейсом BaseCIAdapter. За счет этого архитектура приложения не зависит от конкретной платформы.

Реализация адаптера для GitHub Actions использует клиент `httpx.AsyncClient` с пулом соединений. Фрагмент кода отправки команды на повторный запуск конвейера (Re-run) приведен в листинге 3.3.

```

import httpx
from fastapi import HTTPException

class GitHubAdapter:
    def __init__(
        self,
        token: str,
        base_url: str = "https://api.github.com"
    ):
        self.base_url = base_url
        self.headers = {
            "Authorization": f"Bearer {token}",
            "Accept": "application/vnd.github+json",
            "X-GitHub-API-Version": "2022-11-28"
        }

    async def rerun_workflow(
        self,
        repo_full_name: str,
        run_id: str
    ) -> bool:
        url = (
            f"{self.base_url}/repos/{repo_full_name}"
            f"/actions/runs/{run_id}/rerun"
        )
        async with httpx.AsyncClient(timeout=10.0) as client:
            resp = await client.post(url, headers=self.headers)
            if resp.status_code == 201:
                return True
            raise HTTPException(
                status_code=resp.status_code,
                detail=f"Ошибка внешнего API: {resp.text}"
            )

```

Рисунок 3.3 – Асинхронный метод перезапуска пайплайна через GitHub Actions API

3.4.2 Отображение состояния сборок на клиентской части

Главный экран мониторинга (DashboardPage) агрегирует данные обо всех активных пайплайнах. Статусы сборок нормализуются к единому формату перечисления (QUEUED, RUNNING, SUCCESS, FAILED, CANCELED) вне зависимости от исходного провайдера.

Каждая карточка содержит визуальные индикаторы:

- зеленый индикатор — конвейер выполнен успешно;

- красный пульсирующий бейдж — зафиксирована ошибка выполнения стадии;
- синий анимированный индикатор — пайплайн находится в процессе выполнения;
- контекстная кнопка действия «Re-run» — активна для упавших или отмененных сборок. При нажатии вызывается мутация данных через REST API, а статус немедленно переходит в состояние ожидания запуска.

3.5 Реализация системы логирования, хранения и обработки данных запусков

3.5.1 Сбор и нормализация журналов выполнения

При обнаружении сбоя в работе конвейера система выполняет запрос к внешнему API для получения вывода консоли упавшего шага. Ввиду того, что полный объем журналов сборки может достигать десятков мегабайт, в базе данных PostgreSQL сохраняется только информативный срез (последние 200 строк вывода ошибки, извлекаемые регулярными выражениями по ключевым маркерам ERROR, FATAL, FAILED).

На фронтенде для отображения журналов разработан компонент LogViewer. Он использует моноширинный шрифт, темную цветовую схему и поддерживает автоматическую прокрутку к месту возникновения критической ошибки.

3.5.2 Агрегация и визуализация аналитических метрик

Для оценки эффективности процессов непрерывной интеграции реализован контроллер аналитики `/api/v1/metrics/summary`. Сервис агрегирует исторические записи из таблицы `pipelines` с применением математических функций PostgreSQL:

$$\text{Success Rate} = \frac{N_{\text{success}}}{N_{\text{total}}} \times 100\%, \quad (1)$$

где N_{success} — количество успешно завершенных сборок за выбранный период, N_{total} — общее число запусков конвейеров.

На клиентской стороне вычисленные показатели отображаются в виде обзорных карточек метрик (KPI widgets) и временных графиков, позволяю-

щих отслеживать среднюю продолжительность сборки и динамику процента падений по дням недели.

Разработанный программный комплекс полностью реализует поставленные функциональные требования, формируя надежный и удобный инструмент централизованного мониторинга сборок.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано веб-приложение для централизованного управления и мониторинга CI/CD-пайплайнов, функционирующее по принципу единого информационного окна («Single Pane of Glass»). Разработанная система решает проблему фрагментации данных при мультирепозиторной разработке и использовании гетерогенных систем автоматизации, объединяя сбор и отображение метрик из платформ GitHub Actions и GitLab CI/CD.

В процессе выполнения работы были решены следующие задачи:

- проведен анализ предметной области и существующих решений автоматизации (GitHub Actions, GitLab CI/CD, Jenkins, Argo CD), сформулированы критерии сравнения и выявлена потребность в специализированном легковесном диспетчере сборок;
- обоснован и выбран технологический стек, включающий библиотеку React и TypeScript для клиентской части, асинхронный фреймворк FastAPI на языке Python для серверной части и реляционную СУБД PostgreSQL для долговременного хранения данных;
- составлены функциональные и нефункциональные требования к системе, разработаны диаграмма вариантов использования (Use Case) и пользовательские истории (User Stories);
- спроектирована реляционная модель данных и физическая схема базы данных в третьей нормальной форме, включающая механизмы индексации и безопасного хранения зашифрованных токенов доступа;
- реализована серверная архитектура веб-приложения с модулем аутентификации на основе JWT, асинхронными HTTP-адаптерами к внешним REST API и алгоритмом симметричного шифрования Fernet;
- разработан интерактивный SPA-интерфейс дашборда на React, обеспечивающий фильтрацию сборок, отображение карточек статусов конвейеров, просмотр консольных журналов ошибок и повторный запуск упавших задач в один клик;

- проведено функциональное тестирование основных пользовательских сценариев и выполнено развертывание сервиса.

Все поставленные в курсовом проекте цели и задачи успешно выполнены. Разработанный программный комплекс демонстрирует стабильную работу, сокращает время реакции инженеров на сбои в пайплайнах и может быть расширен за счет добавления новых адаптеров CI/CD-платформ.

Исходный код проекта опубликован и доступен в репозитории:

- репозиторий проекта: <https://github.com/h3sug0/coursework-web-app-ci-cd>;
- веб-приложение, развернутое на хостинге: <https://cicd-monitor-1gu6.onrender.com>.

СПИСОК ЛИТЕРАТУРЫ

- [1] GitHub Actions Documentation [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.github.com/en/actions> (online; accessed: 2026-03-15).
- [2] GitLab CI/CD Documentation [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://docs.gitlab.com/ee/ci/> (online; accessed: 2026-03-15).
- [3] Jenkins Pipeline Documentation [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.jenkins.io/doc/book/pipeline/> (online; accessed: 2026-03-15).
- [4] Argo CD - Declarative GitOps CD for Kubernetes [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://argo-cd.readthedocs.io/en/stable/> (online; accessed: 2026-03-15).
- [5] React – The library for web and native user interfaces [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://react.dev/> (online; accessed: 2026-03-20).
- [6] FastAPI Documentation [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://fastapi.tiangolo.com/> (online; accessed: 2026-03-20).
- [7] PostgreSQL 16 Documentation [Электронный ресурс] [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/16/index.html> (online; accessed: 2026-03-20).